

DESIGN MANUAL

Compounding Calculator

How the calculator is built: the files, every formula and solver, the data model, the visual system, the Windows installer and the web edition.

Version **1.13** Released **29 Oct 2026** Developed by **Sathiya Sekhar E**

Contents

01 Purpose and principles
Scope, rules the code follows, version history

02 Architecture
Files, modules, the update cycle

03 Calculation model
The yearly schedule, step-up, SIP timing, actuals

04 Return measures
Blended CAGR, IRR, doubling, milestones, review returns

05 Solvers
Goal SIP, required return, course correction, youngster model

06 Data and storage
User record, keys, database file, merging

07 Visual design system
Colour, type, components, charts, layout, print

08 Build and release
Windows installer, web edition, version checklist

09 Testing
Test pages, browser checks, verification rules

10 Limits and extending
Model limits and how to add features

01

Purpose and principles

A planning tool for the Technofunda core + satellite approach: two pots compounding at their own rates, a goal, yearly course correction, and a lesson in starting early.

Rules the code follows

- **No server.** Plain HTML, CSS and JavaScript. It runs from local files or as a web page, offline, with no build step.
- **One engine.** Every number comes from `calculator.js`, which has no access to the page. The interface only reads inputs and draws results.
- **Same timing everywhere.** One cash-flow timing rule (chapter 3) is used for the schedule, the IRR, the goal solvers and the review.
- **Answers are checked by putting them back.** Every solved SIP or return is fed into the schedule again and must reproduce the target.
- **Indian conventions.** Lakh/crore grouping, ₹ throughout, financial years shown as 2024–25.
- **The user owns the data.** Data stays in the browser or in a file the user chooses.

Version history

Version	Date	Changes
1.13	29 Oct 2026	Inflation rate in the tab bar (default 6%); Today's ₹ view that deflates every page and shows real returns; "Target in today's money" goal switch saved with the user; Youngster horizons set on the cards with one control that moves all three; Youngster age from 0 (default 20); step-up 10% and SIP 10 years as defaults; presets renamed, 22 in all, with two inflation presets.
1.12	29 Oct 2026	Presets menu (19 demonstrations in four groups) with a presenter note and Previous / Next; plan pages leave year-end records out while a preset is shown; Youngster lab lump sum, three chosen ages up to 100 and waiting steps of 3, 5 or 10 years; CAGR table rates 12–30% including 23% and 26%; phone layout fix for the waiting cards.
1.11	24 Oct 2026	Help button with the User manual and Design manual inside the app (opens at the current page, `F1`); manuals as PDF; storage pill no longer clips "• unsaved"; version stamps on file addresses so browsers never mix old and new files.
1.10	21 Oct 2026	Users with IDs and goals; optional database file (local or OneDrive); annual review by user; merged Goal & schedule page; spouse inside Core engine; step-up in % or ₹ with switch; SIP stop in years and months; 25/50/75% chart markers; IRR beside Blended CAGR; simplified Youngster lab with its own core/satellite sliders; Reset page with Undo; new defaults; formula fixes; new design; version control.

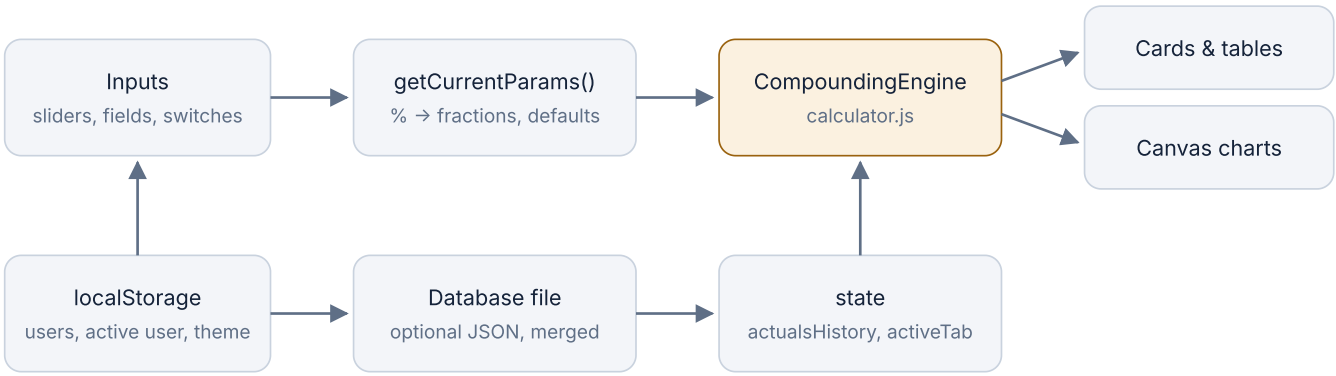
02

Architecture

Files

File	Role
`index.html`	All markup: header, four page panels, modals, icon sprite, print-only report blocks.
`styles.css`	Design tokens (dark and light), components, responsive rules, print styles.
`calculator.js`	The engine, exported as `window.CompoundingEngine` (and as a Node module for tests). No DOM access.
`app.js`	Interface: reads inputs, calls the engine, renders cards, tables and canvas charts; users, storage, database file, modals, Help.
`manual/`	The two manuals (`user-manual.html`, `design-manual.html`), `manual.css`, `manual.js` and screenshots in `img/`.
`test_runner.html`, `test_milestones.html`	Engine test pages (chapter 9).

The update cycle



Every input fires `updateAll()`, which builds the parameter object, calls the engine with the user's year-end records (`planActuals()`: `state.actualsHistory`, or none while a preset is shown), and redraws the Core engine, Goal & schedule, Annual review and spouse comparison. The Youngster lab has its own `updateYoungsterLab()`. The goal, comparison and review solvers rerun the full schedule a few hundred times per update, which is still instant.

Presets

A preset is data, not code: `applyPreset(id)` saves the current inputs for *Undo*, writes the preset's Core engine, goal and Youngster inputs into the form, sets `state.presetId`, opens its page and shows the presenter note. While `state.presetId` is set, `planActuals()` returns no records, so demonstrations are not mixed with the selected user's history; closing the note, Reset page or opening a user clears it. A preset also sets the Today's ₹ view (`real: true` turns it on; otherwise off) and, for goal presets, the goal's inflation switch (`goal.inflationAdjusted`). Presets never save anything.

Engine functions

Function	Returns
`calculateSchedule(params, actuals)`	60-year rows for both pots, horizon summary, IRR, doubling figures, milestones.
`analyzeSpouseImpact(params)`	Self-only vs combined corpus, extra wealth, time saved.
`solveGoal(target, years, params, opts)`	Year the goal is reached, SIP needed, CAGR needed.
`reviewActualsAndCorrect(params, actuals, target)`	Revised projection, return and SIP needed from next year, revised goal year.
`calculateYoungsterWealth(options)`	Yearly rows to the last checkpoint age, values at each checkpoint (`checkpoints`), delay scenarios.
`solveIRR(flows)`	Annual IRR for cash flows at any times.
`steppedMonthly(base, year, stop, mode, pct, amt)`	Monthly SIP in a given year.
`ratesForBlended(b, p)`, `weightedReturnOf(p)`	Core/satellite rates for a blended return; the blended return of a plan.
`requiredSIPAtReturn(...)`, `yearReaching(rows, target, max)`	SIP for a goal at a given return; fractional year a target is first reached.
`formatINR(v, opts)`, `formatPercent`, `formatYears`	Display formatting (lakh/crore grouping; compact L / Cr).

03

Calculation model

Annual steps, two pots, no rebalancing. Year 0 is the start; years 1 to 60 are computed whatever the horizon.

Year 0

```
L      = self lump + spouse lump (if spouse on)
core₀  = L × c            c = core allocation, s = 1 − c
sat₀   = L × s
invested₀ = L
```

Monthly SIP in year y

```
steps = max(0, min(y, stopStepUp) − 1)
pct mode:  M(y) = M₁ × (1 + g)^steps
₹ mode:    M(y) = M₁ + a × steps          year 1 = base SIP; last rise in
year stopStepUp
```

Months of SIP in year y

```
m(y) = 12                if y ≤ N                N = SIP years
     = extra months (0–11) if y = N + 1
     = 0                 after that
A(y) = M(y) × m(y)                               money added in year y
```

Timing weight

SIPs arrive through the year, so they earn only part of the year's return. A full year of monthly SIPs is treated as invested at mid-year; a part-year of *m* months starting in month 1 has its centre earlier.

```
w = 0.5            for a full year (m = 12)
w = 1 − m / 24     for m months (1–11)
This matches the source spreadsheet's semi-annual rule (additions at month 6).
```

Each pot, each year

```
core_add  = A × c
growth_c  = core_open × r_c + (A × w) × c × r_c
core_close = core_open + core_add + growth_c

sat_add   = A × s
growth_s  = sat_open × r_s + (A × w) × s × r_s
sat_close = sat_open + sat_add + growth_s

corpus = core_close + sat_close
invested = invested_prev + A
multiple = corpus / invested
```

Self and spouse money is added together into the same two pots; each keeps its own weight *w*. Nothing moves between the pots.

Recorded years (actuals)

- **Invested**: if recorded, it replaces that year's planned SIPs (split between self and spouse in the planned ratio) with *w* = 0.5.
- **Core / satellite value**: the recorded year-end values replace the model's closes. Growth becomes close − open − added, so the next year starts from the real values.
- Only a total value: it is split in the model's core/satellite ratio for that year.
- Older records with returns instead of values are still honoured (per pot or blended).

Years to next 2× and milestones

Both use log interpolation between year-ends, which is exact for constant growth within a year:

```
year reaching T = (k − 1) + ln(T / V_(k−1)) / ln(V_k / V_(k−1))
k = first year with V_k ≥ T
```

Milestones are multiples (2×, 4×, … up to 131,072×) of the combined lump sum, or of year-1 additions when there is no lump sum.

Spouse comparison

The plan is run twice: spouse off and spouse on. The strip shows both corpora and amounts invested, the extra corpus, and the fractional year when the combined plan reaches the self-only horizon corpus (time saved = horizon − that year).

04

Return measures

Blended CAGR

```
blended = c × r_c + s × r_s          70% × 15% + 30% × 25% = 18%
```

Portfolio IRR

The annual rate *R* that sets the net present value of the plan's cash flows to zero, with each contribution at its real time:

```
flows: −L at t = 0
       −A_self(y)   at t = y − w_self(y)
       −A_spouse(y) at t = y − w_spouse(y)     for y = 1 … horizon
       +corpus      at t = horizon
solve  Σ cf × (1 + R)^(−t) = 0            Newton–Raphson from 15%; bisection
fallback in (−95%, 2000%)
```

With full-year SIPs ($t = y - 0.5$) this equals the spreadsheet's semi-annual IRR annualised, $(1 + r)^2 - 1$. The IRR exceeds the blended CAGR because the pots are not rebalanced: the satellite's share of the money rises every year, so the portfolio's effective rate drifts towards the satellite rate.

Doubling

```
money doubles every   = ln 2 / ln(1 + IRR)
next doubling horizon = years to next 2× measured from the horizon value
rule of 72 (shown for comparison) = 72 / (IRR × 100)
```

Share-of-corpus markers (25 / 50 / 75%)

For *p* in {0.25, 0.5, 0.75}, the chart marks `yearReaching(rows, p × corpus_horizon)`. The headline sentence uses the 50% year and the years left after it.

Today's rupees (inflation-adjusted values)

When `params.realInflation` = $i > 0$, `calculateSchedule` runs the normal (nominal) year loop and then divides every value by the inflation factor at the time it occurs:

```
D(t) = (1 + i)^t
open_r  = open / D(y − 1)        added_r = added / D(y − 0.5)        close_r =
close / D(y)
growth_r = close_r − open_r − added_r            so every row still adds up
invested_r = lump + Σ added_r
IRR flows: cf / D(t)   ⇒   real IRR = (1 + IRR) / (1 + i) − 1   exactly
blended CAGR shown = (1 + c·r_c + s·r_s) / (1 + i) − 1
```

Doubling times, milestones and the 25 / 50 / 75% markers are then computed from the real series, so they are real too. `calculateYoungsterWealth` does the same with `realInflation` and a `yearOffset` (a delayed start of *d* years is deflated from today, not from its own start). The app passes `realInflation` only when the Today's ₹ view is on; the Annual review always runs nominal.

Inflation-adjusted goal

With "Target in today's money" on, the goal solvers run on the plan in today's rupees (`realInflation` = *i*), so the target is compared with real wealth at every year. This is the same as growing the target with inflation, and it gives the year, SIP and CAGR for an inflation-adjusted freedom number. Displayed corpus values still follow the Today's ₹ view. The Annual review compares actual values with the target grown to the deadline: target × (1 + i)^years.

Return achieved in the Annual review

A Modified Dietz return that uses the model's own timing, for the whole portfolio and each pot:

```
growth  = close − open − added
return  = growth / (open + added × w)       w = 0.5 for recorded years
```

Youngster lab mix

Core+SAT doubling time solves `c(1 + r_c)^t + s(1 + r_s)^t = 2` for *t* by bisection; "₹1 L in 30 yrs" is 1,00,000 × the same expression at *t* = 30. Fixed deposit (6.5%) and Nifty 50 (13%) are fixed reference rows.

05

Solvers

All solvers search over the full schedule with monotone bisection (50–60 halvings), so they respect step-up, stop years, months, spouse money and recorded years exactly.

Keeping the core–satellite gap

When a solver changes “the return”, it moves both pots together and keeps the user's spread (satellite − core):

```
spread = r_s − r_c
r_c' = b − s × spread
r_s' = b + c × spread        so c × r_c' + s × r_s' = b exactly
```

Goal: SIP needed on time

1. Set the horizon to the goal deadline and the rates to the assumed blended CAGR (default: the plan's).
2. If the plan with no SIP already reaches the target, the answer is ₹0.
3. Start the upper bound at max(₹10,000, target ÷ (years × 12)) and double it until it reaches the target (cap $₹10^{10}$). If it never does, report “not reachable” (the SIP stops too early).
4. Bisect the starting monthly SIP between 0 and the bound.

Goal: CAGR needed with current SIP

Bisect the blended return between −50% and 150%; outside that range report “over 150%” or “below −50%”.

Goal: year reached

`yearReaching` on the current plan's 60-year rows; null means not within 60 years.

CAGR comparison table

Blended rates 12, 15, 18, 20, 23, 26 and 30%, plus the plan's and the assumed rate. For each: rates by the gap rule, the full schedule (with actuals), IRR, doubling time, corpus at the horizon, goal year and the SIP needed on time.

Annual review: course correction

Output	Method
Revised projection	Schedule with all recorded years, plan after the latest one.
Return needed from here	Bisect the blended return for the remaining years (recorded years fixed), −50% to 150%.
SIP needed, **scale** mode	Used while planned SIP remains. Multiply every future planned SIP by a factor *k* (keeping step-up and stop year) and bisect *k*. Shown as *k* × next year's planned SIP.
SIP needed, **restart** mode	Used when the planned SIP has ended. A flat monthly SIP from next year to the goal year, bisected.
Revised goal year	`yearReaching` on the revised rows.

Past years are never changed by a correction. The goal horizon (not the Core engine horizon) is used throughout this page.

Youngster lab

Same yearly rule as the Core engine, with the tab's own three sliders (core allocation, core CAGR, satellite CAGR) as two pots without rebalancing.

```
lump L at the current age:  core₀ = L × c,  sat₀ = L × (1 − c),  invested₀ = L
horizons h₁ < h₂ < h₃ (years from now, set on the cards; h₃ ≤ 100 − age)
checkpointAges = age + h;  targetAge = age + h₃
horizon = targetAge − current age
checkpoints[i] = { age, corpus, invested, multiple }   null corpus if the age is not after today
```

The horizons live in `state.youngHz`. The master control moves all three by the same number of years; each card's −/+ moves one, kept in order by `fitHorizons()`. Delay scenarios rerun the model from age + *w*, + 2*w* and + 3*w* (*w* = 3, 5 or 10, chosen on the page) with the same lump sum and SIP settings (one level only), and report the loss at the last age in rupees and %.

06

Data and storage

User record

```
{
  "userId": "sekhar",  "name": "Sekhar",
  "createdAt": "2024-04-01T05:00:00Z",  "updatedAt": "2026-04-05T05:00:00Z",
  "startYear": 2024,
  "params": { "selfLump": 100000, "selfMonthly": 10000, "selfStepUpOn": true,
              "selfStepUp": 10, "selfStepUpMode": "pct", "selfStepUpAmt":
1000,
              "selfStepUpStop": 10, "selfAddStop": 10, "selfAddStopMonths": 0,
              "coreAlloc": 70, "coreRet": 15, "satRet": 25, "horizon": 30,
              "spouseInclude": false, … },
  "goal": { "target": 100000000, "years": 20, "assumedReturn": null,
"inflationAdjusted": false },
  "actualsHistory": {
    "1": { "invested": 120000, "coreValue": 168000, "satValue": 84000,
           "recordedAt": "2025-04-03T05:00:00Z" }
  }
}
```

Stored params use percentages as shown on screen; `getCurrentParams()` converts them to fractions for the engine. User IDs match `^[a-z0-9][a-z0-9._-]{1,31}$`. Older profile shapes are upgraded by `normaliseUsers()` on load.

Browser storage keys

Key	Holds
`compounding_calculator_user_profiles_v6`	All users, keyed by user ID.
`compounding_calculator_active_profile_v5`	The selected user ID.
`compounding_calculator_deleted_users_v1`	Deletion markers (user ID → time), so deleted users do not come back when merging.
`compounding_calculator_inflation`	The inflation rate in % (default 6).
`compounding_calculator_real_view`	`1` while the Today's ₹ view is on.
`compounding_calculator_theme`	`dark` or `light`. The manuals read it too.
IndexedDB `compounding-calculator / kv / dbHandle`	The linked database file's handle.

Database file

```
{ "app": "Compounding Calculator", "format": 2, "appVersion": "1.13",
  "savedAt": "…", "users": { … }, "deleted": { "old.user": "2026-…" } }
```

- Created or opened with the File System Access API (Chromium browsers: Edge, Chrome).
- Every save rewrites the whole file 300 ms after the last change.
- On opening, the file and the browser copy are merged: the newer `updatedAt` wins per user; a deletion marker newer than a record removes it.
- After a browser restart the permission must be granted again; the storage pill becomes *Reconnect <file>*.
- Export and import use the same format and the same merge.

Defaults

Where	Values
Core engine and new users	₹1,00,000 lump, ₹10,000 SIP, step-up on 10% (₹1,000 in ₹ mode), last step-up year 10, SIP 10 yrs 0 months, 70/30, 15% / 25%, horizon 30, spouse off.
Goal	₹10 Cr in 20 years, assumed return = plan's blended CAGR.
Youngster lab	Age 20 (range 0–55), no lump sum, ₹10,000, step-up 10% (₹500 in ₹ mode), stop step-up 10, stop SIP 10 yrs 0 months, 70% / 17% / 27%, horizons 30 / 40 / 50 years, waiting steps of 3 years.
Inflation	6% a year; Today's ₹ view off; goal "Target in today's money" off.

07

Visual design system

A ledger at night: deep navy paper, saffron gold for money and the horizon, and one colour per kind of money so a figure's colour says what it is before you read it.

Colour roles

Colours are tokens in `styles.css` on `:root` (dark, the default) and `:root[data-theme="light"]`. Components only use tokens.

`--gold`
#F2B544 · light
#A56A12
Money, horizon, primary action

`--growth`
#3ED6A0 · light
#0C7F5B
Corpus, gains, on track

`--core`
#7E9CFF · light
#3A55C4
Core pot

`--sat`
#2BC3D8 · light
#0A7D8F
Satellite pot

`--rose`
#FF7A8A · light
#BF3A4E
Loss, late, short

`--invested`
#8193AE · light
#8592A6
Money put in

`--bg`
#0D1626 · light
#D5DCE5
Page

`--panel`
#142035 · light
#E7ECF2
Cards

`--text-main`
#EAF0F8 · light
#18283D
Text (secondary #AAB8CD, muted #7A8BA4)

The light theme is a soft paper grey with no pure white, to reduce glare; every accent is darkened to keep contrast on it.

Type

₹22.91 Cr

Bricolage Grotesque, 800 — headline figures, page and card titles

Monthly SIP · Last step-up in year · Portfolio IRR 20.51%
IBM Plex Sans 400–700 — interface text, labels, tables

- `--font-display`: Bricolage Grotesque → Segoe UI Variable Display → Segoe UI → system.
- `--font-ui`: IBM Plex Sans → Segoe UI → system. Body 15px.
- All numbers use tabular figures so columns line up.
- The Einstein line in the header is gold, on one line, with "— Albert Einstein".

Shape and spacing

Radii: 20px (modals, hero), 14px (cards), 10px (inputs, tiles), 7px (chips). Hairlines are translucent blue-grey (`--line`, `--line-strong`). Focus is a 3px gold ring.

Components

Component	Use
Hero band	Corpus in display type, full rupees, multiple chip, invested vs compounding bar; IRR and Blended CAGR side by side on the right.
Split strip	Core and satellite corpus at either end, proportional bar between.
KPI cards	Label, value, one-line explanation. Value colour follows the money role.
Switch, % / ₹ toggle, stepper	On/off for step-up and spouse; mode toggle; – / + for 1-year steps.
Chips	Quick values (₹5 Cr … ₹100 Cr; ₹5k … ₹50k).
Ladder rungs	Doubling milestones; inside the horizon highlighted, beyond it dimmed.
Tables	Horizon row highlighted, right-aligned tabular numbers, sideways scroll inside the card on narrow screens.
Pills	Version (gold outline) and storage status (dot: grey browser, green file, gold warning).
Modals and toasts	Users, database, About, Print, Help. Toasts confirm actions and can carry *Undo*.
Presets menu and note	Menu under **Presets** in the tab bar: groups with a gold heading, numbered items with a one-line description, arrow-key navigation. The presenter note is a gold-outlined strip under the tabs with the preset number, title, talking point, ‹ Previous / Next › and ×.
Help viewer	Large modal with a User / Design switch and the manual in a frame, opened at the current page's chapter.

Charts

- Drawn on `<canvas>`, sized for the device pixel ratio (`prepCanvas`), colours read from the CSS tokens (`palette()`) so both themes match.
- Growth chart: total (growth, filled), core, satellite, invested (dashed); linear or log scale; gold 25/50/75% markers; tooltip lines End goal, % achieved by year, Core, Satellite, Invested, plus Self only with a spouse.
- Review chart: original plan, actual + revised, dashed goal line with its label.
- Redrawn on resize and theme change.

Layout and screens

Width	Behaviour
≥ 1200 × 860	One-screen layout: the Core engine fits the window (sidebar scrolls on its own); other pages scroll inside their panel.
≤ 1500	Header quote hidden; storage pill narrower; Youngster grids drop to 2–3 columns.
≤ 1280	Hero stacks; IRR/CAGR move under the corpus.
≤ 1080	Inputs move above the results (in two columns); review and Youngster lower sections stack.
≤ 760	Header stacks; everything in one column; 14px text.

Print

Print styles switch to light colours, hide controls, and show a report header (investor, date, horizon, version, “Developed by Sathiya Sekhar E”), the chart and a milestones table; the full report adds the 60-year schedule with repeated headers.

Accessibility

Labels on every field, aria-labels on sliders and icon buttons, visible focus, `prefers-reduced-motion` respected, colour backed by words and signs (“short of the goal”, +/−) for the key states.

08

Build and release

Windows installer

Two small Go programs, cross-compiled for 64-bit Windows:

- **Compounding Calculator.exe** (launcher): opens `app\index.html` in Edge (`--app=file:///…`), else Chrome, else the default browser.
- **CompoundingCalculator-Setup.exe**: embeds the launcher, icon and the whole `app\` folder (including `manual\`) with `embed.FS`. Per-user install, no admin.

```
# icon and manifest resource (version in manifest.xml)
rsrc -manifest ../manifest.xml -ico ../app_icon.ico -arch amd64 -o
rsrc_windows_amd64.syso
# build
GOOS=windows GOARCH=amd64 CGO_ENABLED=0 go build -trimpath -ldflags "-s -w -H
windowsgui"
```

Install folder

`%LOCALAPPDATA%\Programs\Compounding Calculator`

Shortcuts

Start menu and desktop, created by PowerShell.

Uninstall entry

`HKCU\…\Uninstall\CompoundingCalculator`, publisher Sathiya Sekhar E, with `Uninstall.exe /uninstall` (and `/quiet`).

Data folder

`%USERPROFILE%\Compounding Calculator Data` with a README, for a local database file. Not removed on uninstall.

Web edition

The same four files plus `manual/`, published as a web page. The page adds a small adaptation layer before `calculator.js`:

- Hides CSV, PDF, schedule CSV and Export users (downloads and printing are blocked there).
- Turns off the database file (no file pickers).
- Replaces `confirm()` with “click again within 6 seconds”, and `alert()` with a message bar.
- Defaults to the dark theme.

Release checklist

1. `APP_INFO` in `app.js`: version, date, history entry.
2. Static fallbacks in `index.html` (`data-app="version"` and `data-app="date"`).
3. The version stamp on every file address: `?v=` on `styles.css`, `calculator.js` and `app.js` in `index.html`, the `app-version` meta tag, and `manual.css`, `manual.js` and images in both manuals. Browsers keep files they have seen before; a new stamp makes them fetch the new version instead of mixing old and new.
4. `appVersion` in the setup program and `assemblyIdentity version` in `manifest.xml`.
5. Version and date on both manuals; rebuild the PDFs.
6. Run the tests (chapter 9), rebuild the installer, republish the web edition.

09

Testing

Test pages

`test_runner.html` runs eight engine suites and must end with “ALL 8 TEST SUITES PASSED”:

Suite	Checks
A	Full plan: invested and corpus in range, milestones present.
B	Monthly SIP off.
C	Pure SIP, lump sum 0.
D	Everything zero.
E	Goal planner with SIP off.
F	Spouse impact with SIP off.
G	Review with SIP off.
H	Youngster with zero monthly.

`test_milestones.html` prints Youngster checkpoints at 50, 60 and 70.

Verification rules

- **Put the answer back**: a solved SIP or return is run through the schedule and must reproduce the target (agreement about 0.001%).
- **Spreadsheet parity**: `solveIRR` with mid-year flows equals the semi-annual IRR (`solveSemiAnnualIRR`) annualised.
- **Limits**: 100% core at 20% in the Youngster lab equals the single-rate model (₹491.80 Cr at 70 for the defaults).
- **Browser check**: open every page in Chromium at desktop and phone widths, both themes; no console errors; no sideways scrolling; Reset page and Undo on every page; database file round trip; print preview.

10

Limits and extending

Model limits

- Constant returns every year; no volatility or sequence risk except through recorded years.
- No rebalancing between core and satellite (by design, matching the method).
- No inflation, tax, costs or withdrawals.
- Annual steps with a mid-year rule for SIPs, not monthly compounding.
- Data lives per browser unless a database file is used; the database file needs a Chromium browser.

Adding an input to the plan

1. Add the control to `index.html` with a stable `id` and a label.
2. Cache it in `els`, read it in `getCurrentParams()` and listen for `input` → `updateAll()`.
3. Use it in `calculateSchedule()` and add a default there.
4. Add it to `defaultPlanParams()`, `readInputsForProfile()` and `applyParamsToInputs()` so it saves, loads and resets.
5. Add a test case and update both manuals.

Adding a preset

Presets live in `PRESET_GROUPS` in `app.js`. Each item has an `id`, `title`, `sub` (menu line), `tab` to open, `note` (talking point) and any of: `core` (built with `corePreset({...})`, Core engine inputs in screen units), `goal` ({target, years}) and `young` (built with `youngPreset({...})`). Add the item to a group and to the table in the User manual. Check its figures with the engine before using it in a talk.

Adding a page

Add a `tab-btn` with `data-tab` and a matching `tab-panel`; give it an update function called from `updateAll()`; add its case to `resetPage()`; add an entry to `HELP_SECTION` and a chapter with that id in the User manual.

Conventions

- Engine values are fractions (0.15); the interface shows percentages.
- Format money only with `formatINR`.
- Colours only from tokens; new colours get dark and light values.
- Wrap every storage access in try/catch; the app must work with storage blocked.
- Plain sentences on screen: say what happens and what to do.